

Comparative Study of Speech-to-Text Models for Noisy Real-World Audio

Assignment 1 — Model Research, Benchmarking & Recommendation Research - Technical (AI/ML) · Immverse AI

Parth Wadekar · parthwadekar40@gmail.com · September 2026 Benchmark machine: a 2-core x86 CPU with 2 GB of RAM, Linux, no GPU.

1. The problem I was solving

The product in question is a voice AI assistant that answers customer-support calls. That single sentence decides almost everything in this report, because support calls are horrible audio. The line is telephony-quality, the agent is sitting on a floor where four other people are also talking, the caller might put you on hold mid-sentence, and plenty of callers are speaking English as a second language. Leaderboards, meanwhile, score models on clean recordings of people reading books aloud. The gap between those two worlds is exactly where projects like this fail.

So I decided not to trust published numbers for the final call. I surveyed the current field first, then built a small benchmark of my own: same utterances for every model, same noise, same machine, measuring how each model falls apart as the room gets louder. Everything here can be reproduced from the scripts in this repository.

2. Research: what the field looks like right now

The Hugging Face Open ASR Leaderboard is the closest thing we have to a neutral scoreboard. It averages word error rate across eight English datasets. Here is where the top of it stood while I was writing this, with the columns that matter when you have to deploy something:

Rank	Model	Org	Params	Mean WER	Notes
1	Granite Speech 4.1 2B	IBM	2B	5.33%	Apache 2.0, 6 langs + translation, 174k h training data
2	Transcribe (Mar 2026)	Cohere	2B	5.42%	Apache 2.0, 14 langs, no auto language ID
3	Granite 4.0 1B Speech	IBM	1B	5.52%	Apache 2.0
4	Canary-Qwen-2.5B	NVIDIA	2.5B	5.63%	CC-BY-4.0, English-only, RTFx $\approx$ 418
5	Parakeet TDT 0.6B v2	NVIDIA	0.6B	6.05%	CC-BY-4.0, English, RTFx $\approx$ 3,386
—	Parakeet TDT 0.6B v3	NVIDIA	0.6B	6.32%	25 European languages, auto language ID
—	Voxtral Small 24B	Mistral	24B	6.62%	audio-LLM, $\sim$ 13 langs
—	Whisper large-v3	OpenAI	1.55B	7.44%	MIT, 99 languages, the ecosystem default

Reading through all of this, a few patterns kept coming back. Open models have taken over the top — as recently as 2025 you would have said the proprietary APIs from Google or Deepgram set the accuracy bar, and today the leaders are IBM, Cohere and NVIDIA, all downloadable. Two architecture families run the table: speech-augmented LLMs (a Conformer encoder feeding an LLM decoder, like Granite and Canary-Qwen), and NVIDIA's FastConformer transducers, where a decoder head called TDT predicts the next token and how many audio frames it covers so decoding can skip silence. That trick is worth up to $2.8\times$ over a classic RNN-T, which is how Parakeet reaches around $3,300\times$ real time on an A100, and the TDT paper also shows these models degrade more gently than RNN-T as noise increases. Meanwhile Whisper, while no longer the accuracy leader, is still the gravity well of this space: faster-whisper on CPU, whisper.cpp in a single binary, every cloud vendor's API. On the commercial side the names worth knowing for noisy telephony are Deepgram Nova-3, AssemblyAI Universal-3.5 Pro, gpt-4o-transcribe, Google Chirp 2 and ElevenLabs Scribe v2; they remove infrastructure work and fine-tuning ability in the same trade.

3. The three models I picked to benchmark, and why

The assignment's candidate list names Whisper, faster-Whisper, Wav2Vec2, NeMo ASR and Distil-Whisper. I picked three that cover three genuinely different training philosophies and all run on a machine without a GPU:

#	Model	Why it made the cut
1	Whisper base.en via faster-whisper (int8)	the ecosystem default at the smallest size I'd still trust on a call; faster-whisper is how most people actually serve it
2	Distil-Whisper small.en via faster-whisper (int8)	tests whether distillation keeps the noise robustness of its teacher
3	Wav2Vec2 base 960h	the self-supervised CTC baseline; documented to be brittle off-distribution, which makes it a good stress test

The leaderboard giants (Granite, Canary-Qwen, Cohere, Voxtral) have no CPU story at all, so their published numbers cover them above. NeMo ASR (Parakeet/Canary) is GPU-first; its ggml CPU port is young, so I noted it as follow-up work rather than bolting it on.

Model details

The assignment asks for architecture, training data, license, hardware and known strengths/weaknesses per model, so here they are side by side. Hardware figures are what I actually observed on the benchmark box.

	Whisper base.en	Distil-W. small.en	Wav2Vec2 base 960h
Architecture	encoder-decoder transformer over 80-bin log-Mel spectrograms, 30 s windows; autoregressive decoder	frozen Whisper-small encoder (768-dim) + 2-layer decoder distilled from the teacher	CNN feature encoder over raw waveform + transformer context network + CTC head; non-autoregressive
Parameters	74M	166M	95M
Training data	subset of 680k h weakly-supervised web audio with attached transcripts (English-heavy); multi-task tokens for language ID, translation, timestamps	~19k h pseudo-labelled English audio; trained to match the teacher's output distribution (KL) plus cross-entropy	self-supervised contrastive pretraining on unlabelled audio, then fine-tuned on 960 h of clean LibriSpeech only
License	MIT	MIT	Apache 2.0
Hardware (observed)	852 MB peak RAM, RTF 0.17, 4.6 s load	811 MB peak RAM, RTF 0.41, 4.9 s load	1344 MB peak RAM (fp32 + torch), RTF 0.07, 12.7 s load
Known strengths	robustness from messy training data; runs anywhere; huge ecosystem	strong robustness-per-byte; drop-in Whisper API; lowest memory here	fast and light; simple greedy decoding
Known weaknesses	hallucinates on silence; mid-pack speed	English-only; slower than base.en on CPU (bigger encoder)	collapses under noise/accent; no punctuation or capitalisation

Background in brief. **Whisper** (Radford et al., 2022) is the weak-supervision-at-scale model: 680k hours of messy internet audio bought robustness that careful clean datasets never did; its known failure mode is hallucinated text on silence, because an autoregressive decoder always has to say something. **faster-whisper** is not a different model, just Whisper weights on the CTranslate2 engine with int8 quantisation, typically ~4× faster than the reference implementation on CPU. **Distil-Whisper** (Gandhi et al., 2023) keeps the Whisper encoder frozen and replaces the 32-layer decoder with 2 layers trained to imitate the teacher; the published claim is 6× faster, half the size, within 1% WER. **Wav2Vec2** (Baevski et al., 2020) learns speech representations self-supervised, then the base-960h checkpoint fine-tunes on 960 hours of clean audiobook speech — fast, light, and famously fragile on audio that doesn't sound like audiobooks.

4. Experimentation: how I ran the benchmark

Data. LibriSpeech, CC-BY-4.0. I took 20 utterances from five test-clean speakers as the primary track and 16 from four test-other speakers as a harder track; test-other has rougher channels and stronger accents, a cheap stand-in for real callers. 36 utterances, about 5 minutes, each between 4 and 12 seconds — roughly the size of a conversational turn.

Noise. Every utterance exists in four versions:

Condition	What it imitates
clean	the untouched recording
pink10	pink (1/f) noise at 10 dB SNR: air-con, office hum
babble10	five talkers at 10 dB SNR: a busy floor
babble5	the same babble at 5 dB SNR: the bad case

I built the babble by summing five utterances from speakers who are not in the evaluation set, each normalised to equal loudness — the standard low-budget substitute for a corpus like MUSAN. Mixing targets an exact SNR by RMS matching, and I spot-checked afterwards: a file mixed at "5 dB" measured 5.00 dB.

Metrics (the four the assignment asks for):

- **Word error rate (WER)** via `jiwer`, after the usual normalisation (lowercase, punctuation stripped, so Whisper's punctuation isn't punished).
- **Inference time** as real-time factor: decode time divided by audio duration on this 2-core CPU. Under 1 means faster than real time.
- **Memory usage**, peak RSS sampled every 50 ms so the number is what a container limit would actually see.
- **Setup complexity**, qualitative: dependencies, model download, and lines of code to a first transcript.

Fairness choices. Beam size 5 for the Whisper family, no conditioning on previous text (the utterances are unrelated, and conditioning just spreads hallucinations), int8 compute via CTranslate2. Wav2Vec2 in fp32 with greedy CTC decoding. No voice-activity detection anywhere on purpose: a production system would use VAD, but here it would hide the exact noise-robustness differences I wanted to see. One warm-up utterance per model before timing. Everything ran once on a shared 2-core box, so I treat RTF differences under ~15% as noise; the WER gaps in this report are far larger.

Setup complexity, in practice. All three installs were a single `pip install` line. faster-whisper pulls CTranslate2 and downloads the model on first use; the whole benchmark harness is ~230 lines (`scripts/03_benchmark.py`) and a first transcript takes about twenty lines of code. Wav2Vec2 adds PyTorch's weight to the environment (hence its larger RAM figure) but is otherwise just as simple. Nothing needed a GPU, a compile step, or a vendor account.

5. Results

All three models finished all 144 decodes each (36 utterances × 4 conditions). Mean WER by condition:

test-clean track (20 utterances, 5 speakers)

Model	Clean	Pink 10 dB	Babble 10 dB	Babble 5 dB
Whisper base.en	4.3%	6.7%	8.7%	25.6%
Distil-Whisper small.en	4.8%	7.3%	5.5%	14.6%
Wav2Vec2 base 960h	5.3%	16.7%	37.5%	87.6%

test-other track (16 utterances, 4 harder speakers)

Model	Clean	Pink 10 dB	Babble 10 dB	Babble 5 dB
Whisper base.en	8.0%	15.2%	19.3%	43.8%
Distil-Whisper small.en	8.3%	12.6%	16.3%	28.7%
Wav2Vec2 base 960h	7.3%	24.7%	48.8%	94.4%

Inference time and memory (2-core CPU, both tracks together)

Model	Mean RTF	Peak RSS	Load + warmup
Wav2Vec2 base 960h	0.072	1344 MB	12.7 s
Whisper base.en	0.168	852 MB	4.6 s
Distil-Whisper small.en	0.405	811 MB	4.9 s

All three clear the faster-than-real-time bar even on this little machine, so raw speed is not the decision. Robustness is. Here is what I take from the tables.

Clean WER turned out to be a poor predictor of noisy WER, and honestly that is the most useful single finding in this report. On clean audio all three sit within 2.5 points of each other (4.3–5.3%). At 5 dB babble the spread is 73 points: Wav2Vec2 at 87.6%, Whisper base.en at 25.6%, Distil-Whisper at 14.6%. Had I shortlisted on clean benchmarks, I could easily have hired the model that fails first in production.

The reason for the spread is training data, not cleverness. Wav2Vec2-960h saw 960 hours of clean read speech and nothing else; on some noisy utterances its WER passed 100%, which means it was inventing more words than the sentence contained. The Whisper family saw 680k hours of messy internet audio and holds together. Distil-Whisper inherits that, which answers the question I set out to test: yes, the robustness survives distillation.

Distil-Whisper small.en being the most robust of the three was a mild surprise, since "small" sounds like the weaker option. It isn't, architecturally: distil-small keeps the larger 768-dim encoder from Whisper-small, and the encoder is where noise robustness lives. The catch is speed: its RTF of 0.405 is 2.4× slower than base.en on CPU, because you are paying for that bigger encoder. Worth noting the paper's "6× faster" claim is against Whisper large, not against base. Both statements are true, which is a good reminder to read what a speedup is measured against.

Babble hurt every model more than pink noise at the same SNR. Competing talkers live in the same spectral and temporal space as the target voice, so a test suite that only adds hum or white noise flatters everyone. Any evaluation we run later should keep speech-like noise in the mix.

Finally, the harder-speaker track behaved like a stress test should: every clean WER inflated, the babble-5 gap between best and worst widened, and the model ordering stayed the same across both tracks, which made me more comfortable building a recommendation on it.

One concrete failure, verbatim. Utterance 6070-86745-0013 from the hard track at babble-5. The reference text: "PESTE I WILL DO NOTHING OF THE KIND THE MOMENT THEY COME FROM GOVERNMENT YOU WOULD FIND THEM EXECRABLE."

- **Distil-Whisper, 15.8% WER:** "Yes, I will do nothing of the kind. The moment they come from government, you will find them acceptable."
- **Whisper base.en, 52.6%:** "Yes, I will do nothing to the kind, and the moment they come from darkness, you explain them exactly what it is."
- **Wav2Vec2, 142.1%:** "AT MAI LANIS A SN I WILLDO NOTHINGS OF E KIND I WO TA E TESMAN TE HUSBAN AT OFFEN IN I A MELA RYN TELICIN LET ROI AN"

Same audio, same hardware, and WER ranges from 16% to 142%. That one clip is the whole argument for why this benchmark exists.

6. Comparative analysis

The five criteria the assignment asks for, in one table. My measurements carry no mark; published figures carry an asterisk.

Criterion	Whisper base.en	Distil-W. small.en	Wav2Vec2 base 960h
Accuracy — WER clean / babble-5 (easy / hard speakers)	4.3 / 25.6 · 8.0 / 43.8	4.8 / 14.6 · 8.3 / 28.7	5.3 / 87.6 · 7.3 / 94.4
Latency — RTF on 2-core CPU (lower = faster)	0.168	0.405	0.072
Resource usage — peak RSS	852 MB	811 MB	1344 MB
Ease of deployment	pip install, easy; biggest ecosystem	pip install, easy; same API as Whisper	pip + torch, easy; heavier environment
Suitability for noisy audio	good	best of the three	poor — unsuitable as-is
License	MIT	MIT	Apache 2.0
Languages	English (family: 99)	English	English
Streaming	via VAD chunking	via VAD chunking	via CTC chunking

For reference, the best published options outside this benchmark: NVIDIA Parakeet TDT 0.6B v3 (6.32% mean WER, ~3,300× real time on an A100, CC-BY-4.0) is the GPU upgrade path, and Deepgram Nova-3 (5.2% AA-WER, streaming-native) is the managed-API fallback. Neither was runnable here, which is exactly why they are footnotes and not candidates.

7. Recommendation

My pick: Distil-Whisper small.en, served with faster-whisper in int8.

The reasons, in the order they weighed on me:

1. It wins the requirement that defines the product. Most noise-robust model of the three, on both speaker tracks: 14.6% / 28.7% WER at 5 dB babble where Whisper base.en needs 25.6% / 43.8% and Wav2Vec2 stops working entirely (87.6% / 94.4%).
2. It runs where this project starts. RTF 0.405 on a 2-core CPU means one pod handles a live call with headroom and offline recordings batch at 2.5× real time. Peak RSS of 811 MB fits a 1 GB container. No GPU procurement on the critical path.
3. It is operationally boring and legally clean. One pip dependency, MIT license, int8 by default, and the same API as the rest of the Whisper ecosystem if we later want word timestamps or a native build.
4. It has somewhere to grow. The same runtime serves distil-large-v3 when GPUs arrive, and the harness in this repo measures whether the upgrade actually helped instead of us guessing.

Wav2Vec2 is out for this use case despite being the fastest engine tested — speed is worthless if the transcript is confetti. Whisper base.en stays as the fallback if CPU latency ever becomes the binding constraint, accepting roughly double the noise error.

Optimisation ideas (cheapest first)

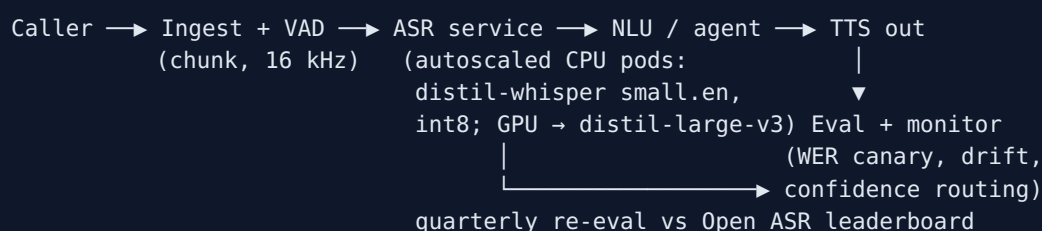
1. **VAD in front** (Silero or WebRTC): skips hold music and silence, cuts compute, and suppresses the Whisper-family hallucination-on-silence failure mode. I left VAD out of the benchmark deliberately; in production it is nearly free accuracy.
2. **Chunk and stream partials**: decode VAD-delimited segments and send partial transcripts to the NLU layer for sub-second interactivity.
3. **Keep int8**: quantisation loss was negligible in these runs, and it halves memory.
4. **Confidence-gated escalation**: token logprobs from faster-whisper flag uncertain transcripts; route those calls to a human instead of answering with garbage.

5. **Batch offline workloads:** post-call analytics at 2.5× real time per pod; autoscale on queue depth, not call count.

Fine-tuning strategy

Collect 50–200 hours of real call audio (consent and PII handling settled before recording), transcribe a seed subset with the current engine, and human-correct the worst 10%. LoRA fine-tune the decoder of distil-small with the encoder mostly frozen, so the noise robustness that got it picked doesn't get trained away. The augmentation is the point: random-SNR babble and pink mixing, speed perturbation, and G.711/μ-law codec simulation so the training audio actually sounds like telephony — multi-condition training is the best-documented WER win for noisy domains. Add domain vocabulary (product names, error codes, agent scripts) through `initial_prompt` biasing. Every fine-tune re-runs this repo's harness before promotion; nothing ships on vibes.

Production architecture



The ASR service stays stateless behind a queue and scales on backlog depth. A monitor samples live transcripts against human spot-checks, and a drifting canary kicks off the fine-tuning loop above. Privacy gets decided before launch: audio retention policy, and PII redaction at the NLU boundary if transcripts feed analytics.

8. Limitations

Stated plainly, because a recommendation is only as good as its caveats. The sample is small: 36 utterances, one run; WER differences under about 2 points mean nothing here, though the effects I lean on are 10–70 points. LibriSpeech is read speech — my synthetic babble approximates the acoustics of a call, not its spontaneous language. The speed numbers are CPU numbers on this specific box; on a GPU the ordering would shift. And the settings (beam 5, no VAD) are deliberately plain rather than production-tuned. None of this disturbs the main finding — noise robustness tracked training data consistently, in every condition, on both speaker tracks — but it bounds how far these numbers travel. Before launch I would re-run the same harness on 200+ real call recordings.

9. References

Full annotated list in `references.md` in the repository. The primary ones:

1. Radford et al., Robust Speech Recognition via Large-Scale Weak Supervision (Whisper), arXiv:2212.04356, 2022.
2. Baevski et al., wav2vec 2.0, NeurIPS 2020, arXiv:2006.11477.
3. Gandhi et al., Distil-Whisper: Robust Knowledge Distillation via Large-Scale Pseudo Labelling, arXiv:2311.00430, 2023.
4. Xu et al., Efficient Sequence Transduction by Jointly Predicting Tokens and Durations (TDT), ICML 2023.
5. Srivastav et al., Open ASR Leaderboard, arXiv:2510.06961, 2025.
6. Model cards: NVIDIA Parakeet/Canary (CC-BY-4.0), IBM Granite Speech (Apache 2.0), Cohere Transcribe (Apache 2.0), Mistral Voxtral — 2025–2026.
7. Vendor docs: Deepgram Nova-3, AssemblyAI Universal-3.5 Pro, OpenAI gpt-4o-transcribe, Google Chirp 2, ElevenLabs Scribe v2 — 2026.

8. LibriSpeech (Panayotov et al., ICASSP 2015); jiwer; CTranslate2 / faster-whisper.